

Solution Code - Module 3-Core Java - TransactionHandlingJDBC

Below is the complete functional source code implementation for this assignment. All logic, validation rules, and structural requirements have been satisfied.

Source File: Q33_TransactionHandlingJDBC.java

```
1 | import java.sql.Connection;
2 | import java.sql.DriverManager;
3 | import java.sql.PreparedStatement;
4 | import java.sql.ResultSet;
5 | import java.sql.Statement;
6 |
7 | public class Q33_TransactionHandlingJDBC {
8 |     private static final String DB_URL = "jdbc:sqlite:bank.db";
9 |
10 |    public static void main(String[] args) {
11 |        System.out.println("=== JDBC Transaction Handling Demo (Money Transfer) ===");
12 |
13 |        try {
14 |            Class.forName("org.sqlite.JDBC");
15 |            initDatabase();
16 |
17 |            System.out.println("\nInitial Balances:");
18 |            printBalances();
19 |
20 |            // Case 1: Successful transfer
21 |            System.out.println("\nAttempting to transfer 200 from Account 1 (Alice) to Account 2 (Bob)...");
22 |            performTransfer(1, 2, 200.0);
23 |            printBalances();
24 |
25 |            // Case 2: Failed transfer (Insufficient balance)
26 |            System.out.println("\nAttempting to transfer 1000 (Insufficient balance) from Account 2 (Bob) to Account 1 (Alice)...");
27 |            performTransfer(2, 1, 1000.0);
28 |            printBalances();
29 |
30 |        } catch (Exception e) {
31 |            e.printStackTrace();
32 |        }
33 |    }
34 |
35 |    private static void initDatabase() throws Exception {
36 |        try (Connection conn = DriverManager.getConnection(DB_URL);
37 |            Statement stmt = conn.createStatement()) {
38 |
39 |            stmt.execute("CREATE TABLE IF NOT EXISTS accounts ("
40 |                + "id INTEGER PRIMARY KEY, "
41 |                + "owner TEXT NOT NULL, "
```

Solution Code - Module 3-Core Java - TransactionHandlingJDBC

```
42 |         + "balance REAL NOT NULL"
43 |         + ");");
44 |
45 |         stmt.execute("DELETE FROM accounts;");
46 |         stmt.execute("INSERT INTO accounts (id, owner, balance) VALUES (1, 'Alice', 1000.0);");
47 |         stmt.execute("INSERT INTO accounts (id, owner, balance) VALUES (2, 'Bob', 500.0);");
48 |         System.out.println("Accounts table initialized with Alice (1000) and Bob (500).");
49 |     }
50 | }
51 |
52 | private static void performTransfer(int fromAccountId, int toAccountId, double amount) {
53 |     String debitSql = "UPDATE accounts SET balance = balance - ? WHERE id = ?;";
54 |     String creditSql = "UPDATE accounts SET balance = balance + ? WHERE id = ?;";
55 |     String checkBalanceSql = "SELECT balance FROM accounts WHERE id = ?;";
56 |
57 |     Connection conn = null;
58 |     PreparedStatement checkStmt = null;
59 |     PreparedStatement debitStmt = null;
60 |     PreparedStatement creditStmt = null;
61 |
62 |     try {
63 |         conn = DriverManager.getConnection(DB_URL);
64 |
65 |         // Disable auto-commit to begin the transaction manually
66 |         conn.setAutoCommit(false);
67 |
68 |         // 1. Verify sufficient funds
69 |         checkStmt = conn.prepareStatement(checkBalanceSql);
70 |         checkStmt.setInt(1, fromAccountId);
71 |         try (ResultSet rs = checkStmt.executeQuery()) {
72 |             if (rs.next()) {
73 |                 double currentBalance = rs.getDouble("balance");
74 |                 if (currentBalance < amount) {
75 |                     throw new Exception("Insufficient balance. Transfer aborted.");
76 |                 }
77 |             } else {
78 |                 throw new Exception("Source account not found.");
79 |             }
80 |         }
81 |
82 |         // 2. Perform Debit operation
83 |         debitStmt = conn.prepareStatement(debitSql);
84 |         debitStmt.setDouble(1, amount);
85 |         debitStmt.setInt(2, fromAccountId);
86 |         debitStmt.executeUpdate();
87 |         System.out.println("Debit completed successfully.");
88 |
89 |         // 3. Perform Credit operation
90 |         creditStmt = conn.prepareStatement(creditSql);
91 |         creditStmt.setDouble(1, amount);
```

Solution Code - Module 3-Core Java - TransactionHandlingJDBC

```
92 |         creditStmt.setInt(2, toAccountId);
93 |         creditStmt.executeUpdate();
94 |         System.out.println("Credit completed successfully.");
95 |
96 |         // Commit transaction if both succeeded
97 |         conn.commit();
98 |         System.out.println("Transaction COMMITTED successfully.");
99 |
100 |     } catch (Exception e) {
101 |         System.out.println("Error occurred: " + e.getMessage());
102 |         if (conn != null) {
103 |             try {
104 |                 // Rollback transaction on failure
105 |                 conn.rollback();
106 |                 System.out.println("Transaction ROLLED BACK successfully.");
107 |             } catch (Exception ex) {
108 |                 System.err.println("Rollback failed: " + ex.getMessage());
109 |             }
110 |         }
111 |     } finally {
112 |         try {
113 |             if (checkStmt != null) checkStmt.close();
114 |             if (debitStmt != null) debitStmt.close();
115 |             if (creditStmt != null) creditStmt.close();
116 |             if (conn != null) conn.close();
117 |         } catch (Exception e) {
118 |             e.printStackTrace();
119 |         }
120 |     }
121 | }
122 |
123 | private static void printBalances() throws Exception {
124 |     try (Connection conn = DriverManager.getConnection(DB_URL);
125 |          Statement stmt = conn.createStatement();
126 |          ResultSet rs = stmt.executeQuery("SELECT * FROM accounts;")) {
127 |         while (rs.next()) {
128 |             System.out.println("  Account " + rs.getInt("id") + " (" + rs.getString("owner") + "):
Balance = " + rs.getDouble("balance"));
129 |         }
130 |     }
131 | }
132 | }
```